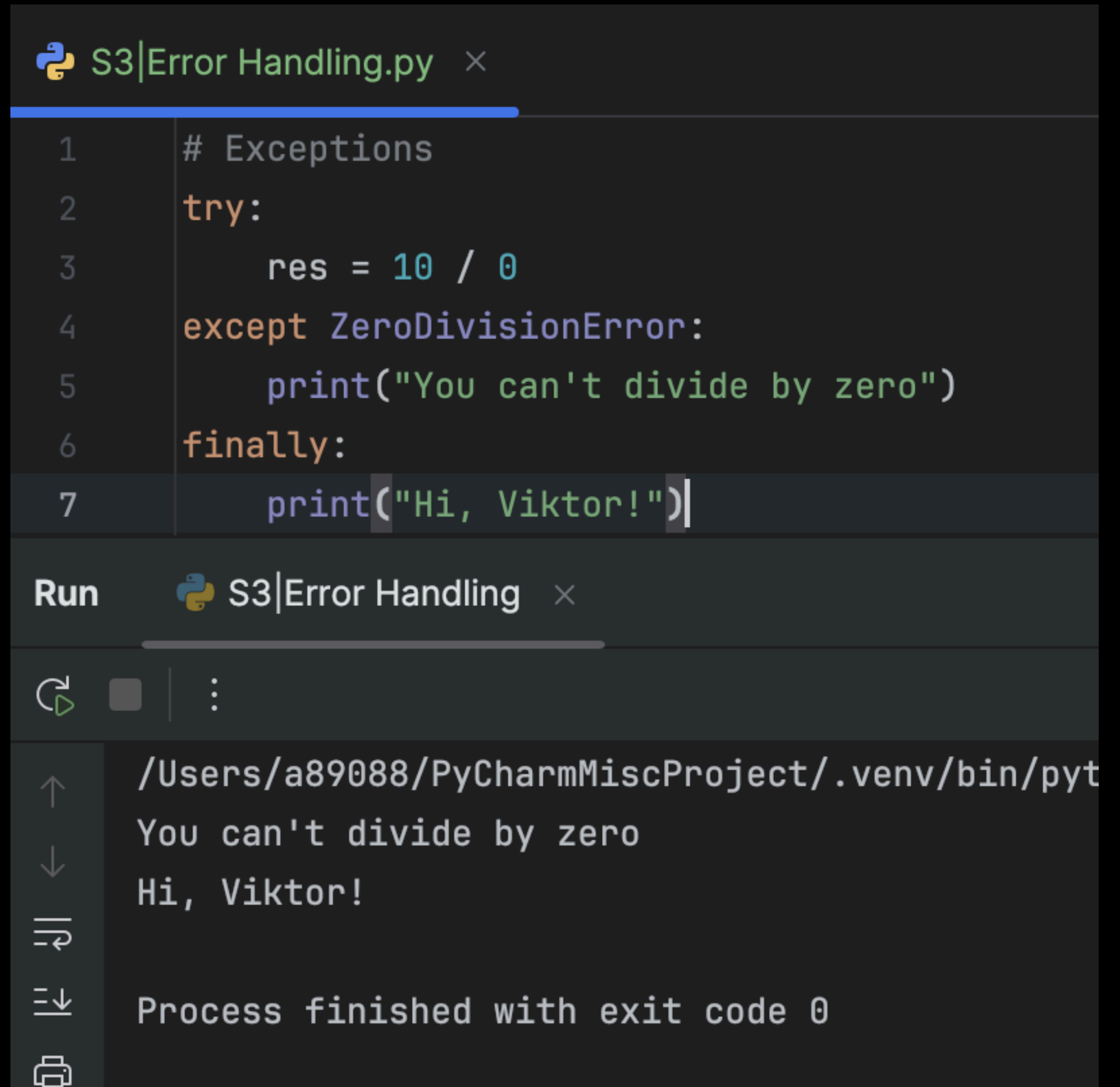


M1S3 | Error Handling & Testing

try-except under the hood

- Exception Object creating
- Handler searching
- Handling(if handler found)
- Program run continuing
- Unhandled Exceptions → Error



```
S3|Error Handling.py ×  
1 # Exceptions  
2 try:  
3     res = 10 / 0  
4 except ZeroDivisionError:  
5     print("You can't divide by zero")  
6 finally:  
7     print("Hi, Viktor!")  
  
Run S3|Error Handling ×  
↑ /Users/a89088/PyCharmMiscProject/.venv/bin/pyt  
↓ You can't divide by zero  
|| Hi, Viktor!  
↺  
≡↓ Process finished with exit code 0  
🖨
```

Error Handling Best Practices

Be concrete

handle concrete exceptions, not everything in a row

Avoid empty exceptions

always understand, which error you want to handle

Use **finally**

helps to free memory, other resources

Write informative messages

in your own exceptions

catch bugs early

better code design

less debugging time

Why write tests?

for Software I mean

higher product quality

faster development cycles

Unit-test

1 module/method

```
import unittest

class TestMathUtils(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-1, -4), -5)

    def test_is_even_true(self):
        self.assertTrue(is_even(4))

    def test_is_even_false(self):
        self.assertFalse(is_even(3))

unittest.main(argv=['first-arg-is-ignored'],
```

Integration test

multiple components

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"msg": "Hello"}
```

✓ 0.0s

```
from fastapi.testclient import TestClient

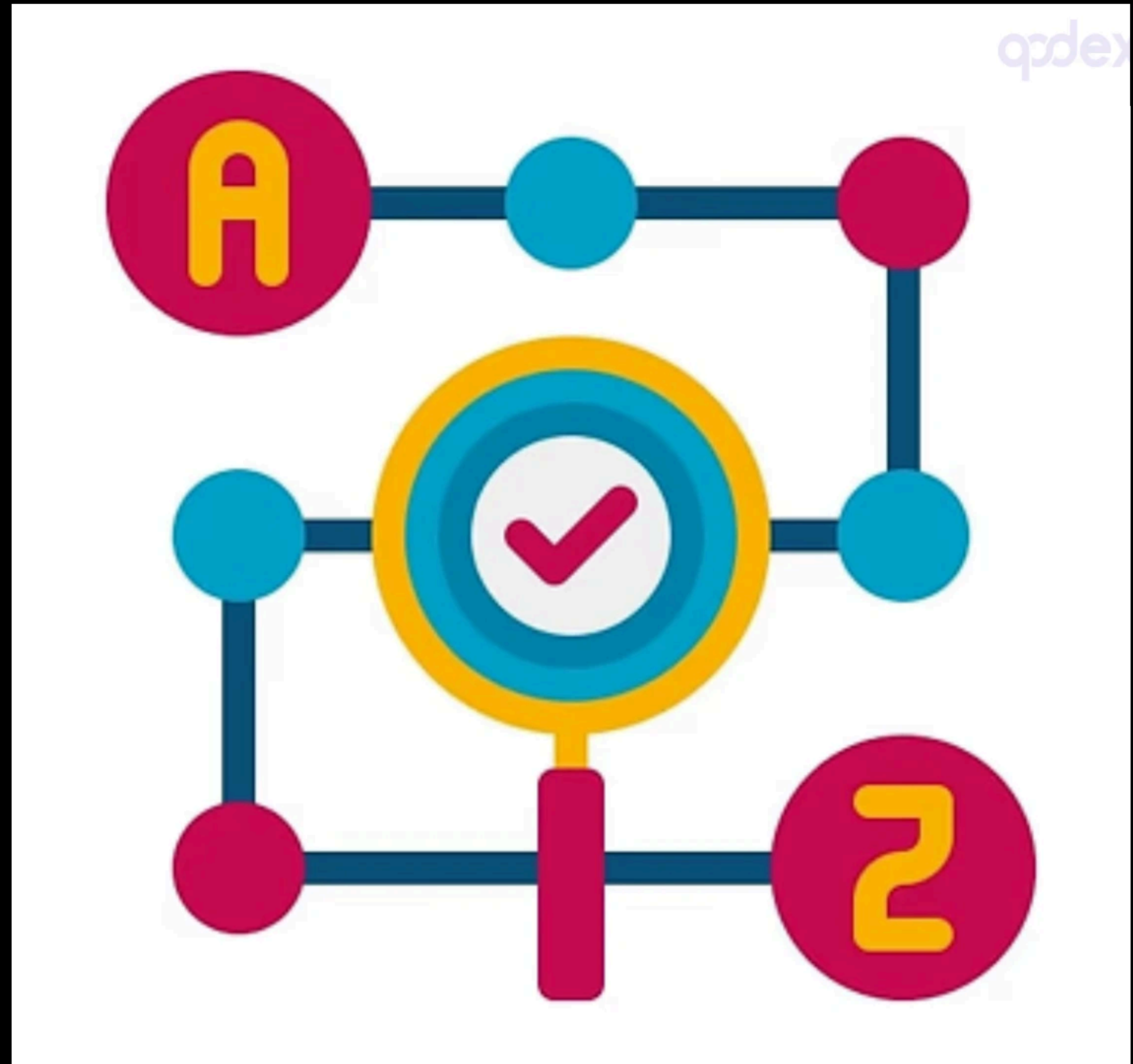
client = TestClient(app)

def test_root():
    r = client.get("/")
    assert r.status_code == 200
    assert r.json() == {"msg": "Hello"}
```

✓ 0.0s

E2E tests

for user's scenarios/use cases



AAA

test-code organization pattern

- Arrange - preparing the env
- Act - testing act
- Assert - result checking